Supervised learning

Artificial Intelligence
CSC 480
Rodrigo Canaan
rcanaan@calpoly.edu

Adapted with permission from
Julian Togelius
julian.togelius@nyu.edu

Types of learning

- **Supervised learning**
  Learning to predict or classify labels based on labeled input data

- **Unsupervised learning**
  Finding patterns in unlabeled data

- **Reinforcement learning**
  Learning well-performing behavior from state observations and rewards

Types of learning

- **Supervised learning**
  Learning to predict or classify labels based on labeled input data

- **Unsupervised learning**
  Finding patterns in unlabeled data

- **Reinforcement learning**
  Learning well-performing behavior from state observations and rewards

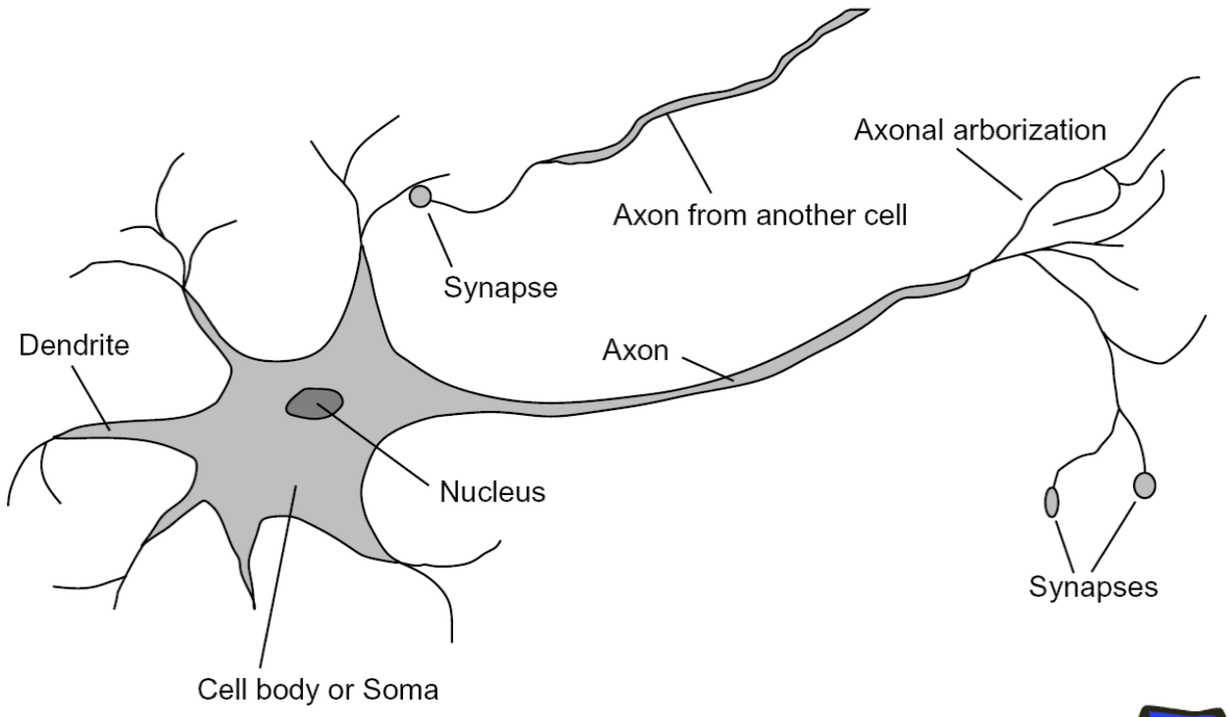
Perceptrons

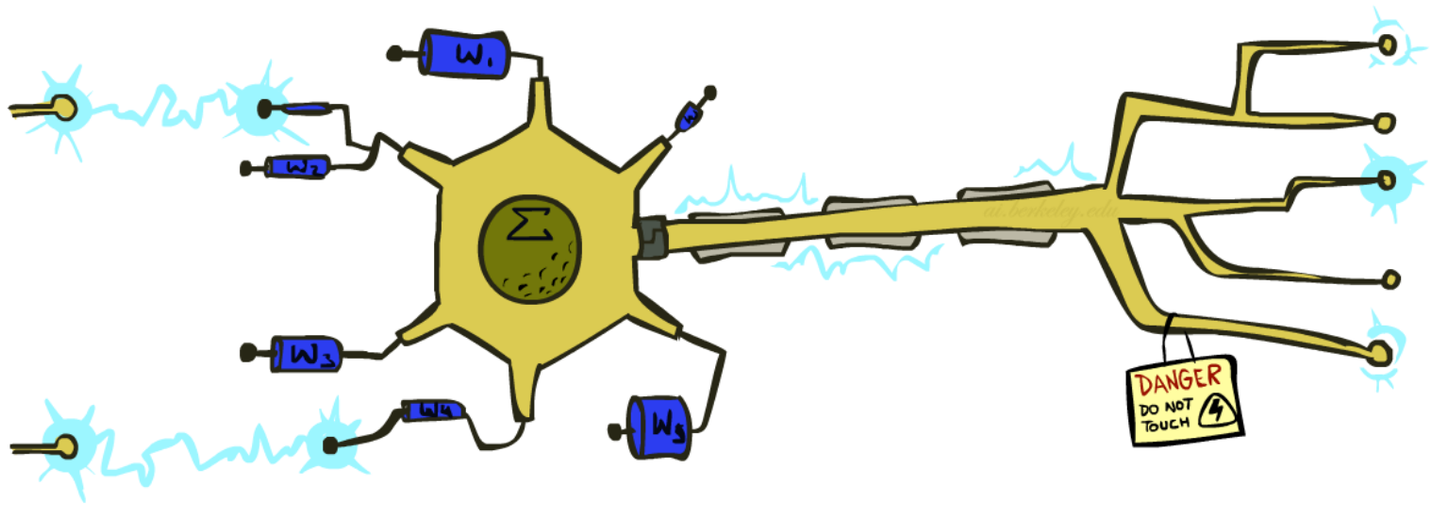
- Early attempt at “neural networks” - copying neurophysiology to produce a learning machine

- Very simple and fast learning algorithm for linearly separable problems

- The basis for many more advanced neural network variants, such as Multilayer Perceptrons (and, by extension, deep networks)

Very loose biological analogy



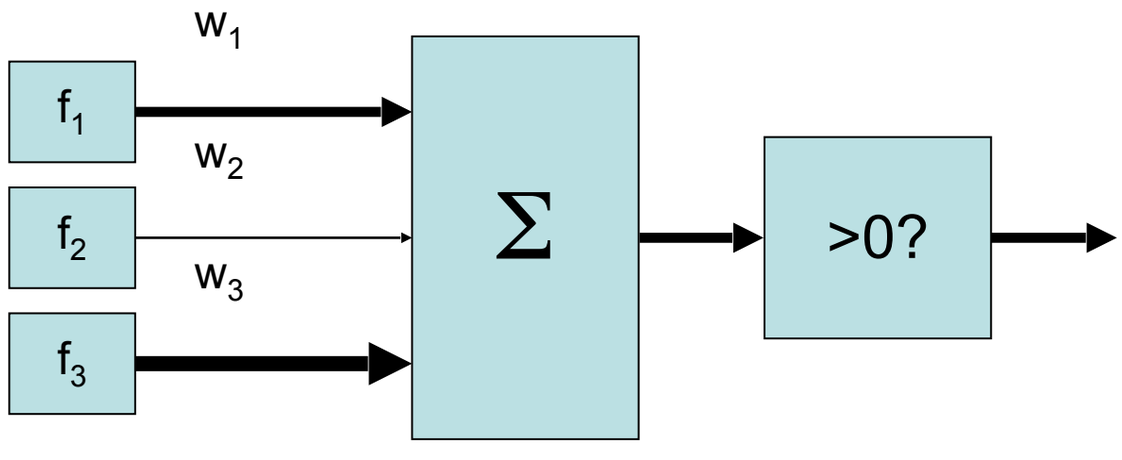


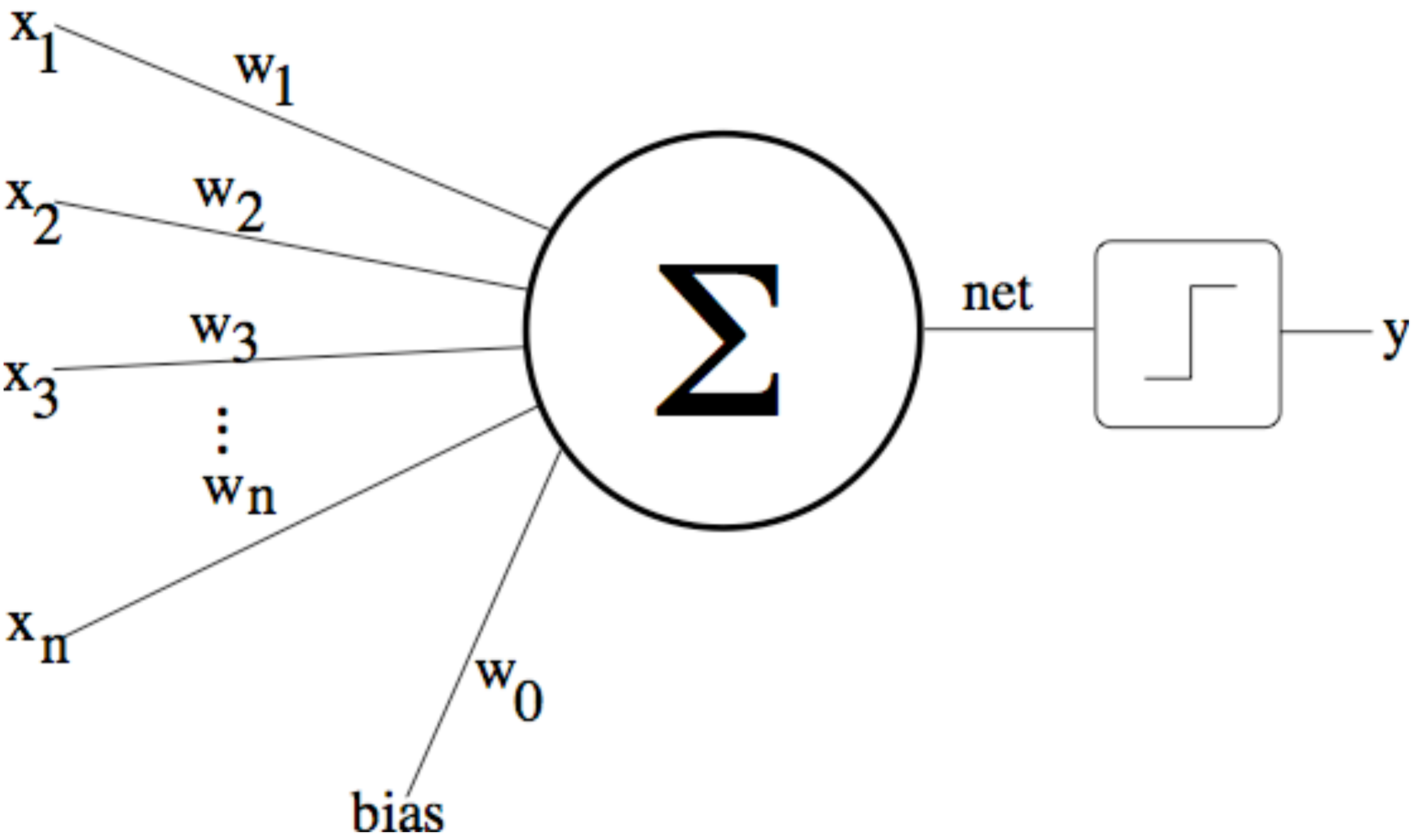
Perceptrons are linear classifiers

- Inputs are feature values
- Each feature has a weight
- Sum is the activation

$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

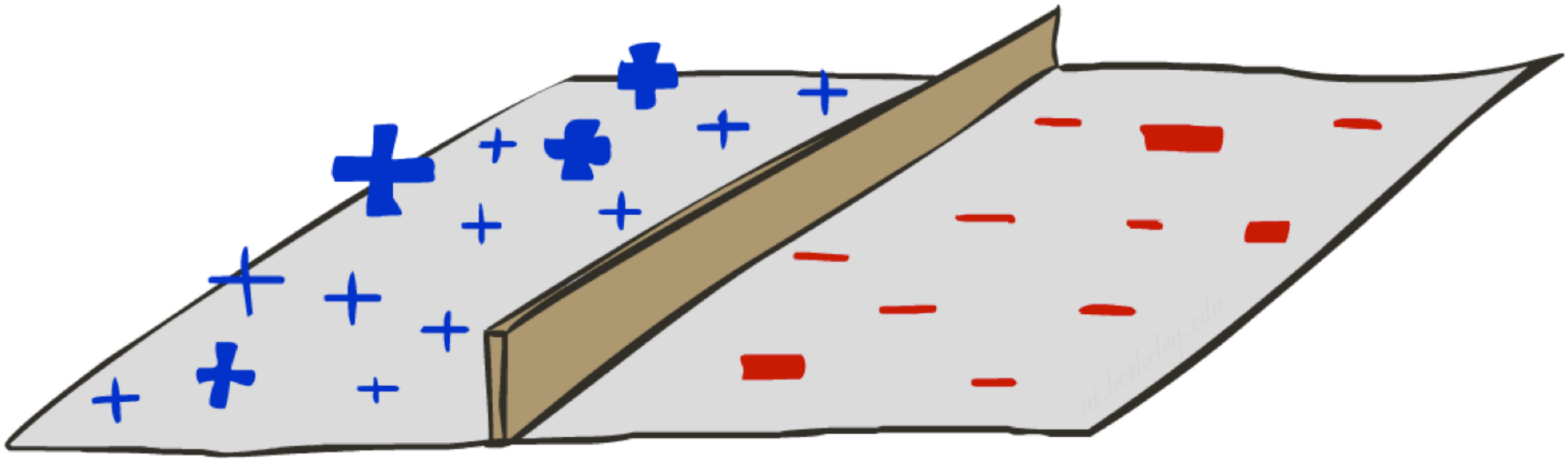
- If the activation is:
  - Positive, output +1
  - Negative, output -1





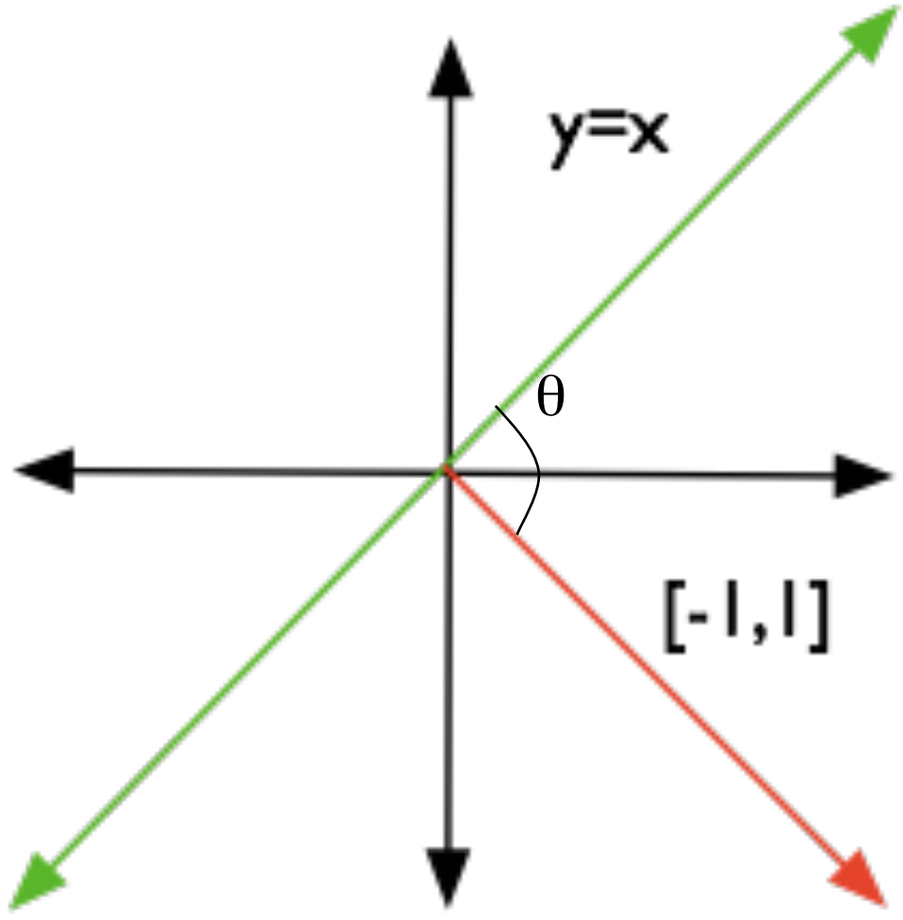
$$f(x) = \begin{cases} 1 & \text{if} \quad \sum_{i=1}^{n} w_i x_i + b > 0 \\ 0 & \text{else} \end{cases}$$

Decision surface



Decision surface

$$y = x$$
$$0 = x - y$$
$$0 = [1, -1] \begin{bmatrix} x \\ y \end{bmatrix}$$

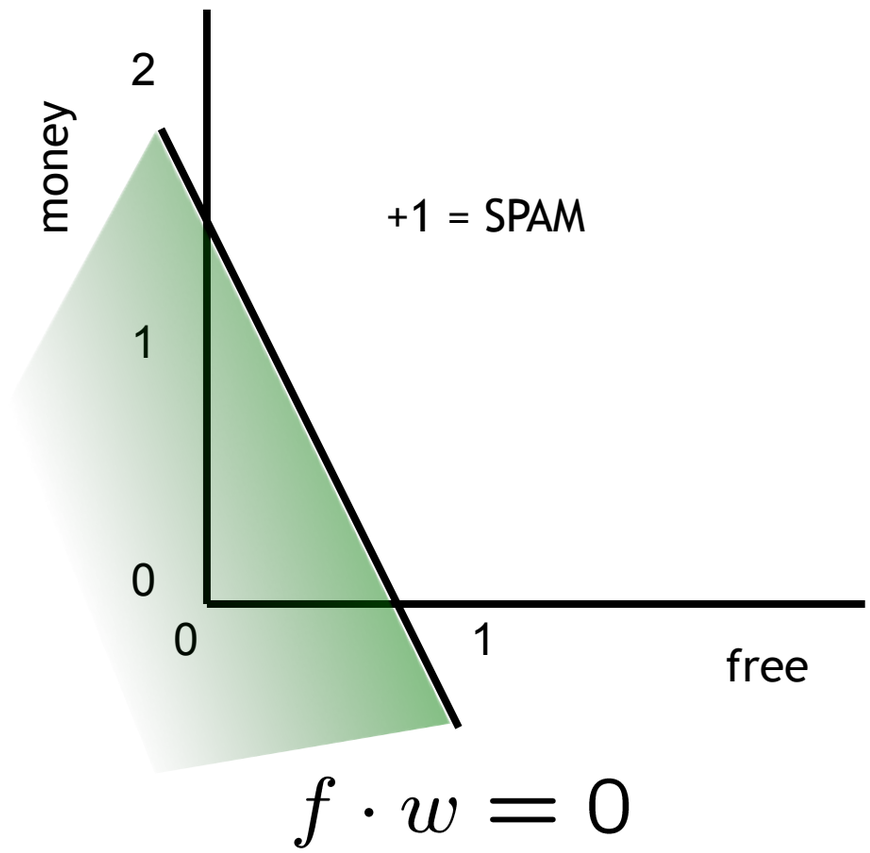


In general a **hyperplane** is defined by

$$0 = \vec{w} \cdot \vec{x}$$

Model usage

- In the space of feature vectors
  - Examples are points
  - Any weight vector is a hyperplane
  - One side corresponds to Y=+1
  - Other corresponds to Y=-1



Model training



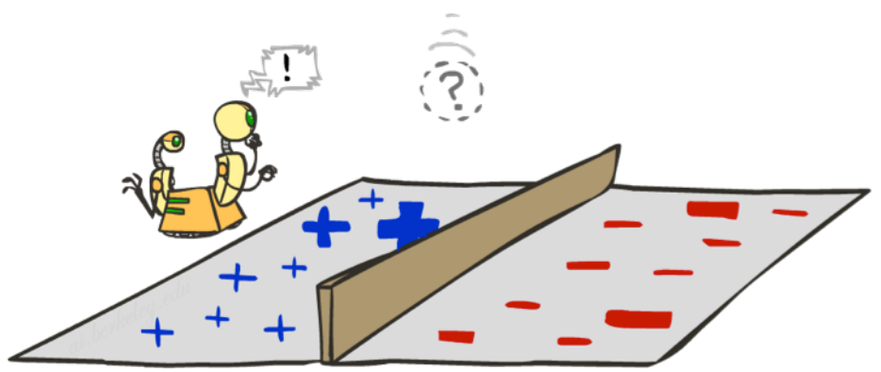
Algorithm

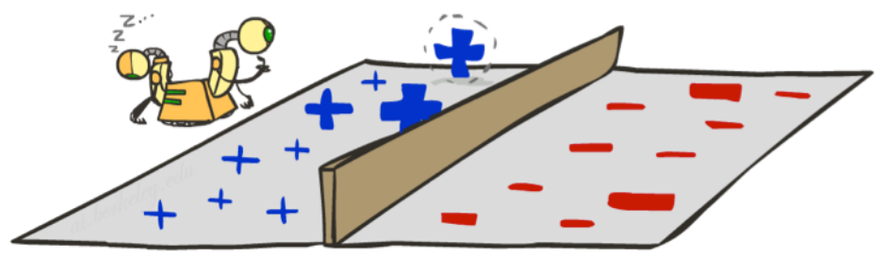
- Start with random weights.
  For each training instance:

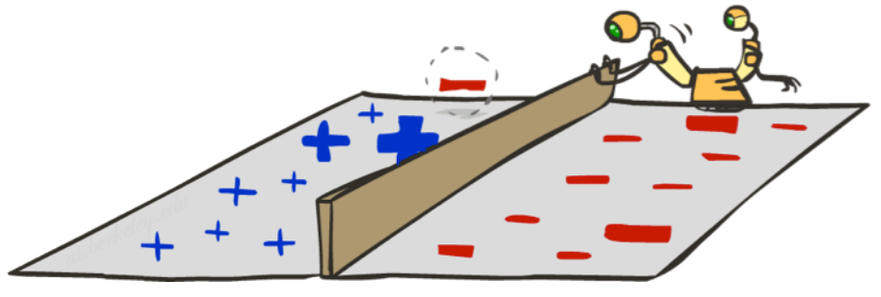
- Classify with current weights

- If correct (i.e., y=y*), no change!

- If wrong: adjust the weight vector







Algorithm - Linear Classifier

Input : list of n training examples $X^{(i)}, y^{(i)}$

where $\forall i : X^{(i)} = (x_1^{(i)}, x_2^{(i)}),\ y^{(i)} \in \{+1,-1\}$,

learning rate η

Output : A linear classifier W, b

Algorithm :

Randomly initialize W and b;

While makes errors on training set **do**

for $(X^{(i)}, y^{(i)})$ **do**

$\mathrm{h} = W^{\top} X^{(i)} + b$

$L^{(i)} = \mathrm{loss}\,(y^{(i)}, \mathrm{h})$

$$W = W - \eta * \frac{\partial L^{(i)}}{\partial W},\ b = b - \eta * \frac{\partial L^{(i)}}{\partial b}$$

end

end

Perceptron Loss

- For binary classification, the classification error (0-1 loss) is discontinuous, and has no useful gradient.
- A point is correctly classified when $y^{(i)}$ and $h^{(i)}$ have the same sign.
    - In other words if $y^{(i)}h^{(i)} > 0$
- Define the loss for a point i be $L^i(W, b) = \max(0, -y^{(i)}h^{(i)})$
    - Loss is *positive* if we misclassified and 0 otherwise
    - Loss is *larger* if h is of high magnitude. Large h means the model puts the point far from the decision boundary
        - Loss punishes “confidently wrong” classifications

Perceptron Loss

- Expanding h, we get
  $L^i(W, b) = \max(0, -y^{(i)}(W^\top X^{(i)} + b))$

- The gradient is therefore:

- $$\frac{\partial L^{(i)}}{\partial W} = \begin{cases} 0 & \text{if } y^{(i)} h^{(i)} > 0, \\ -y^{(i)} X^{(i)} & \text{if } y^{(i)} h^{(i)} \leq 0, \end{cases}$$

- $$\frac{\partial L^{(i)}}{\partial b} = \begin{cases} 0 & \text{if } y^{(i)} h^{(i)} > 0, \\ -y^{(i)} & \text{if } y^{(i)} h^{(i)} \leq 0. \end{cases}$$

Algorithm - Linear Classifier

Input : list of n training examples $X^{(i)}, y^{(i)}$

where $\forall i : X^{(i)} = (x_1^{(i)}, x_2^{(i)}),\ y^{(i)} \in \{+1,-1\}$,

learning rate η

Output : A linear classifier W, b

Algorithm :

Randomly initialize W and b;

While makes errors on training set **do**

for $(X^{(i)}, y^{(i)})$ **do**

$\mathrm{h} = W^\top X^{(i)} + b$

if $y^{(i)} h \leq 0$ // misclassified or at boundary

$W = W + \eta y^{(i)} X^{(i)}$

$b = b + \eta y^{(i)}$

end

end

end

Algorithm - Linear Classifier

Input : list of n training examples $X^{(i)}, y^{(i)}$

where $\forall i : X^{(i)} = (x_1^{(i)}, x_2^{(i)}),\ y^{(i)} \in \{+1,-1\}$,

learning rate η

Output : A linear classifier W, b

Algorithm :

Randomly initialize W and b;

While makes errors on training set **do**

for $(X^{(i)}, y^{(i)})$ **do**

$h = W^\top X^{(i)} + b$

if $y^{(i)} h \leq 0$ // misclassified or at boundary

$W = W + \eta y^{(i)} X^{(i)}$

$b = b + \eta y^{(i)}$

end

end

end

This is the negative of $\frac{\partial L^{(i)}}{\partial W}$

Algorithm - Linear Classifier

Input : list of n training examples $X^{(i)}, y^{(i)}$

where $\forall i : X^{(i)} = (x_1^{(i)}, x_2^{(i)}),\ y^{(i)} \in \{+1,-1\}$,

learning rate η

Output : A linear classifier W, b

Algorithm :

Randomly initialize W and b;

While makes errors on training set **do**

  for $(X^{(i)}, y^{(i)})$ **do**

    $h = W^\top X^{(i)} + b$

    if $y^{(i)} h \leq 0$ // misclassified or at boundary

      $W = W + \eta y^{(i)} X^{(i)}$

      $b = b + \eta y^{(i)}$

    end

  end

end

This is the negative of $\frac{\partial L^{(i)}}{\partial b}$

Algorithm - Linear Classifier

Input : list of n training examples $X^{(i)}, y^{(i)}$

where $\forall i : X^{(i)} = (x_1^{(i)}, x_2^{(i)}),\ y^{(i)} \in \{+1,-1\}$,

learning rate η

Output : A linear classifier W, b

Algorithm :

Randomly initialize W and b;

While makes errors on training set **do**

One Epoch

for $(X^{(i)}, y^{(i)})$ **do**

$\mathrm{h} = W^\top X^{(i)} + b$

if $y^{(i)} h \leq 0$ // misclassified or at boundary

$W = W + \eta y^{(i)} X^{(i)}$

$b = b + \eta y^{(i)}$

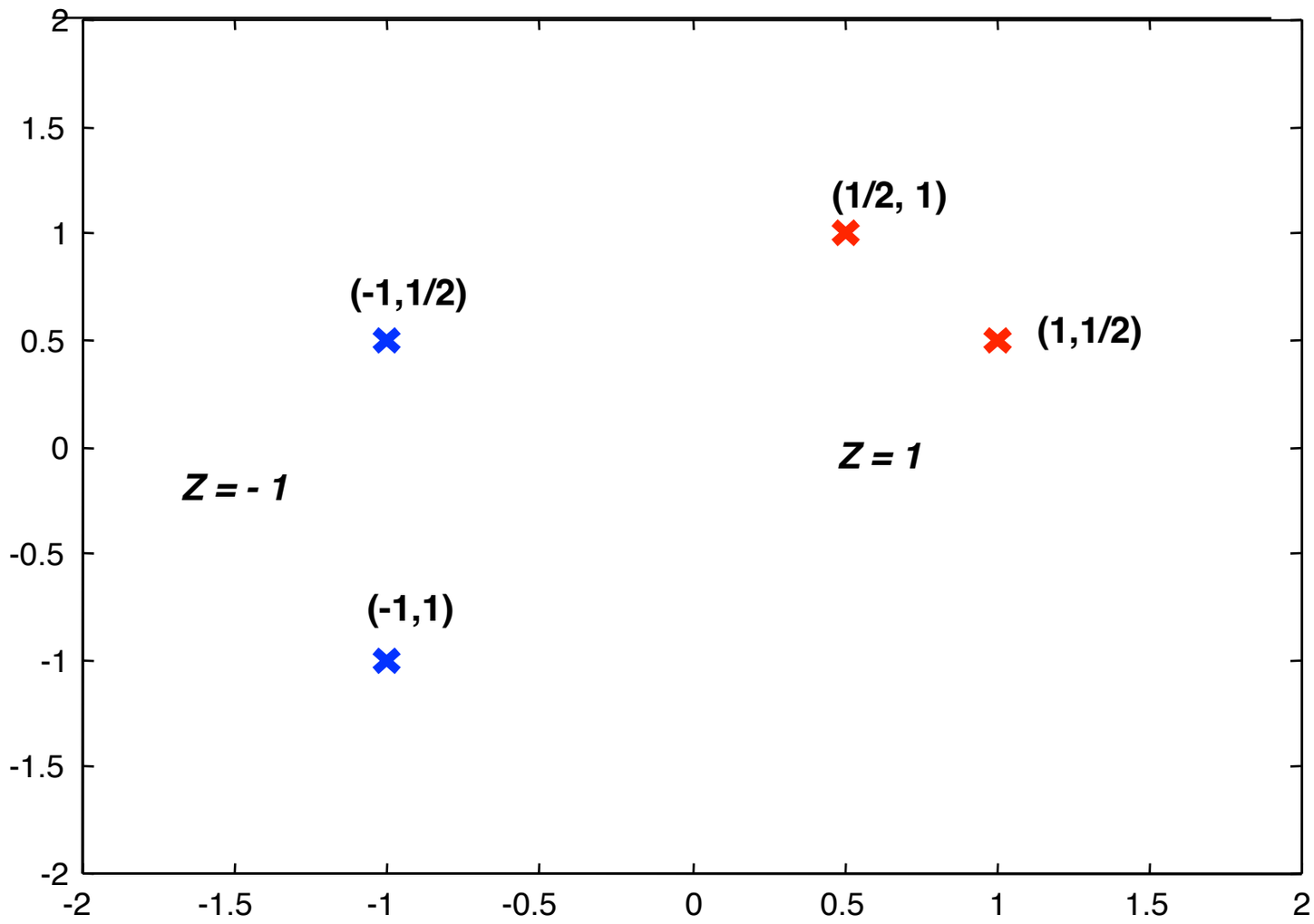
end

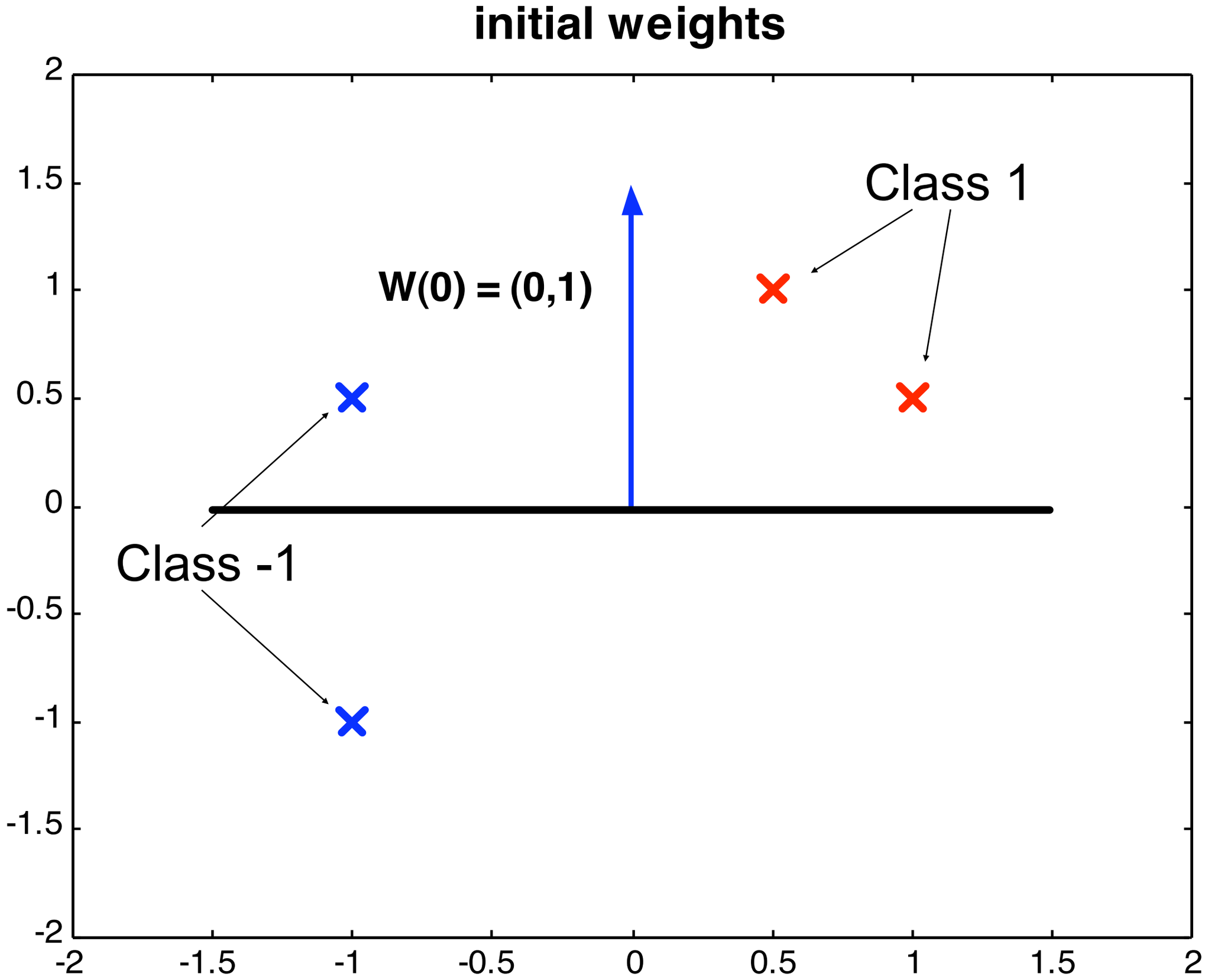
end

end

A simple example

4 linearly separable points




initial weights
W(0) = (0,1)
Class 1
Class -1
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2

Updating Weights

Upper left point (-1,1/2) is wrongly classified.

Let's use η = 1/6, and let's ignore the bias for now

X = (-1,1/2)

Y = -1

w = (0,1)

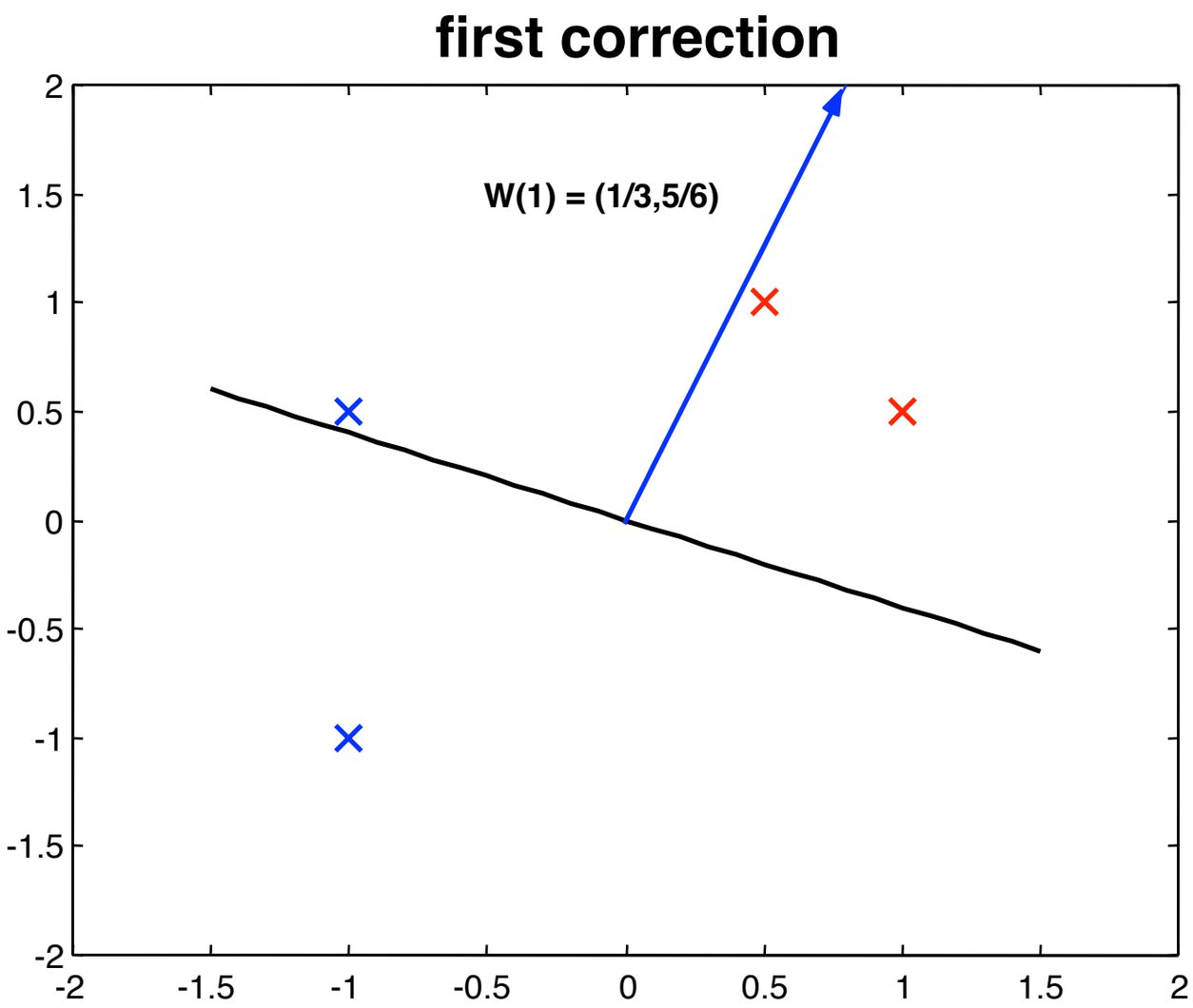
h = 0*(-1) + 1*(1/2) = 1/2 = 1

w = w + η*(*y* -h)*x

= (0,1) + 1/6*(-1-1)*(-1,1/2)

= (0,1) - 1/3 (-1,1/2)

= (1/3, 5/6)


first correction
W(1) = (1/3,5/6)
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2

Updating Weights, Ctd

Upper left point is still wrongly classified

X = (-1,1/2)

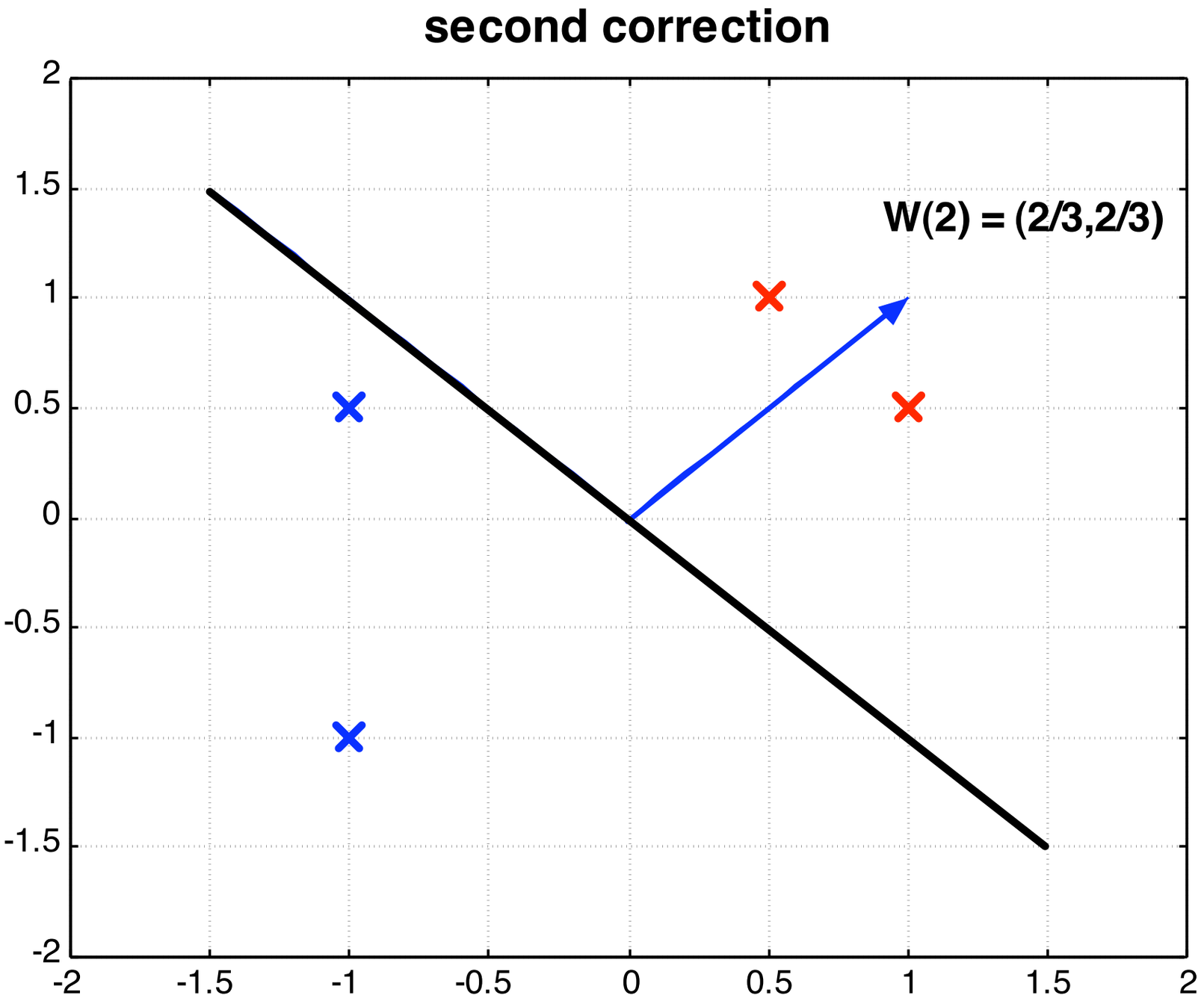
Y = -1, h = 1

w = w + η*(*y* -h)**x*

= (1/3,5/6) + 1/6*(-2)*(-1,1/2)

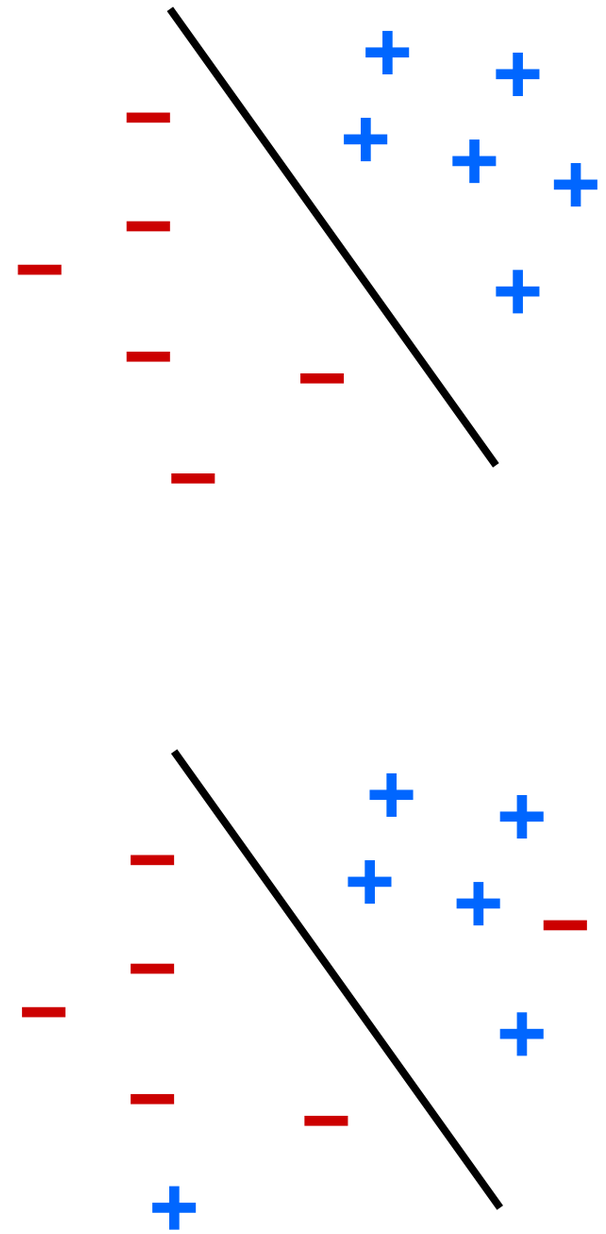
= (1/3,5/6) - 1/3 (-1,1/2)

= (2/3, 2/3)


second correction
W(2) = (2/3,2/3)
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2

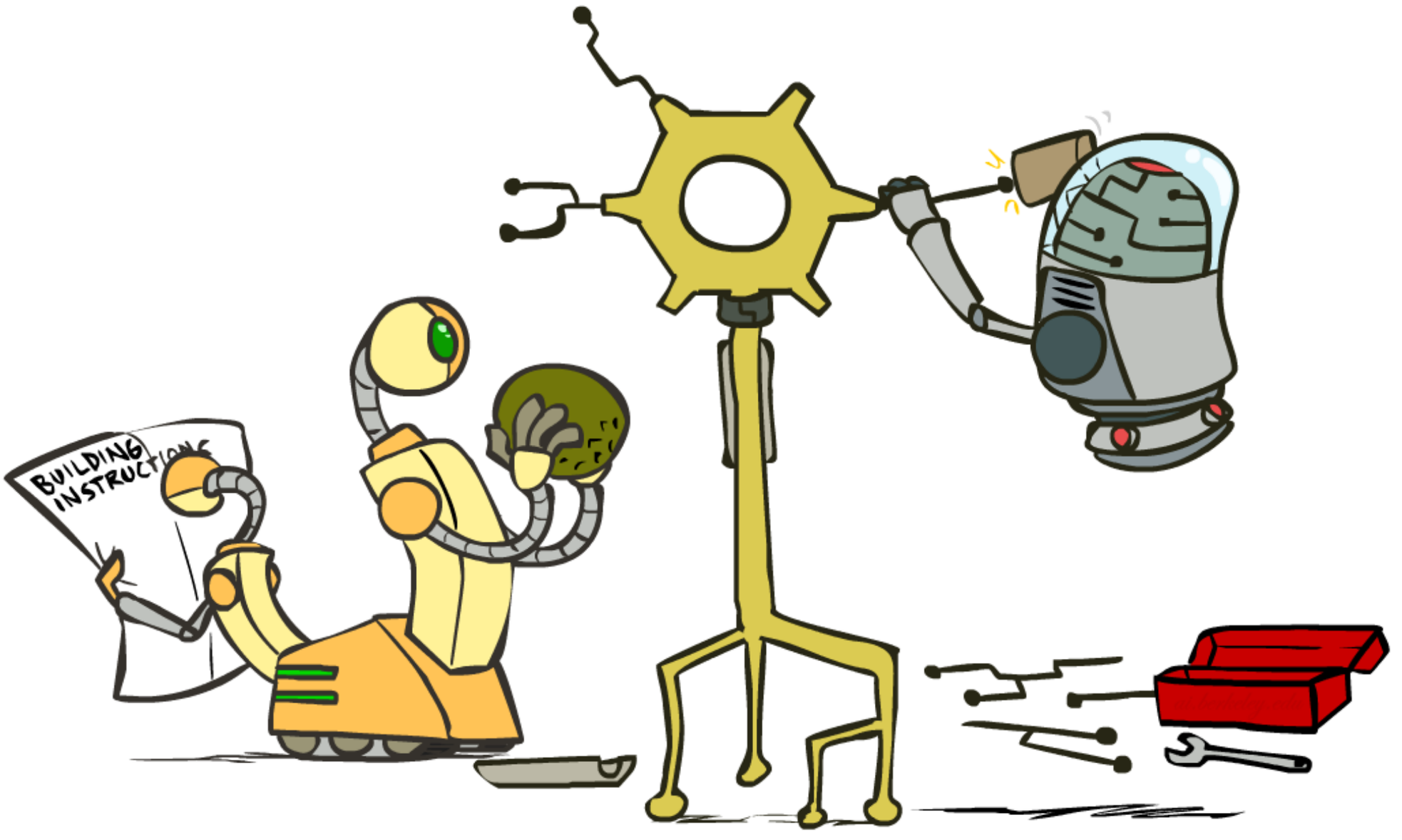
Convergence of Perceptrons

- We say the data is **separable** if some parameters get the training set perfectly correct



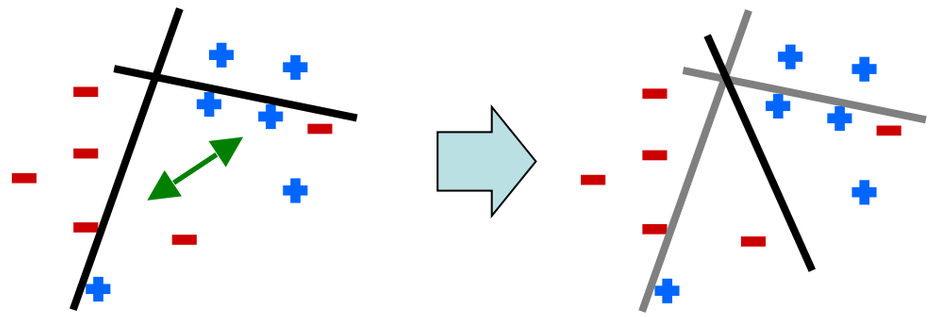
- **Convergence**: if the training is separable, perceptron will eventually converge (binary case)

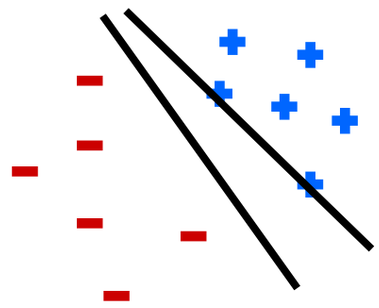
Improving the perceptron

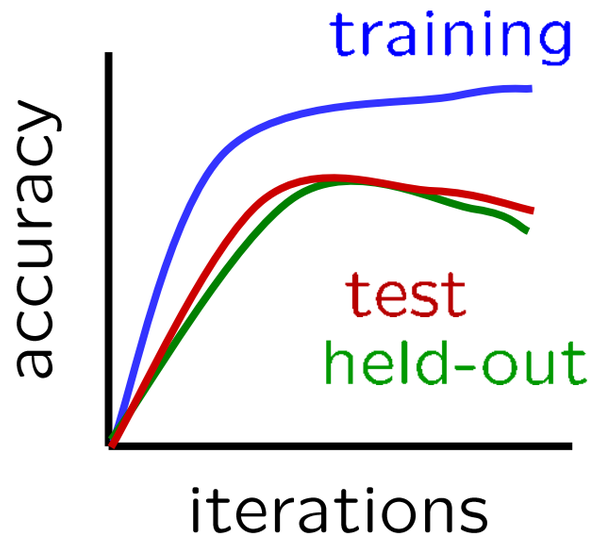


Perceptron problems

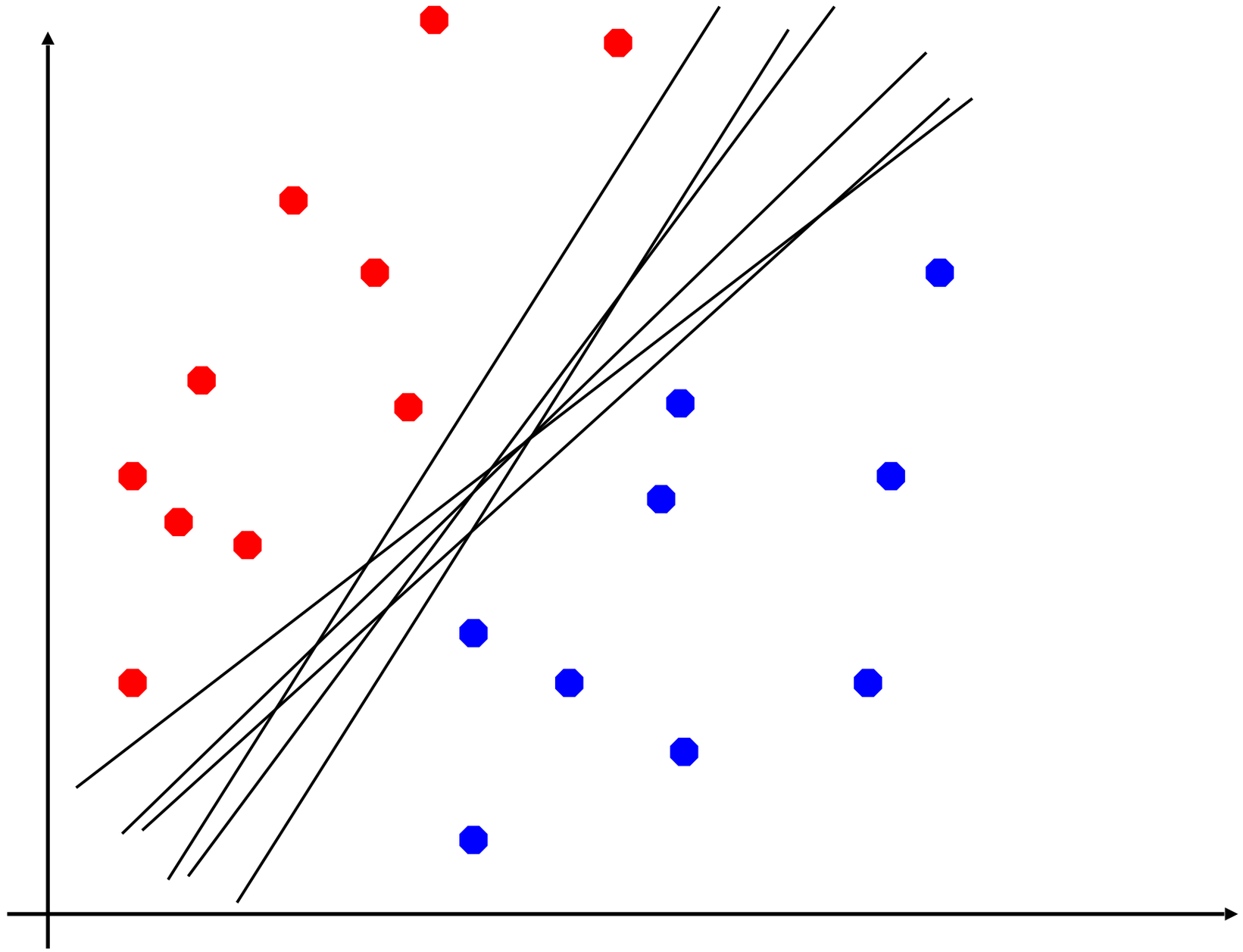
- Noise: if the data isn't separable, weights might thrash
  - Averaging weight vectors over time can help (averaged perceptron)
- Mediocre generalization: finds a "barely" separating solution
- Overtraining: test / held-out accuracy usually rises, then falls
  - Overtraining is a kind of overfitting





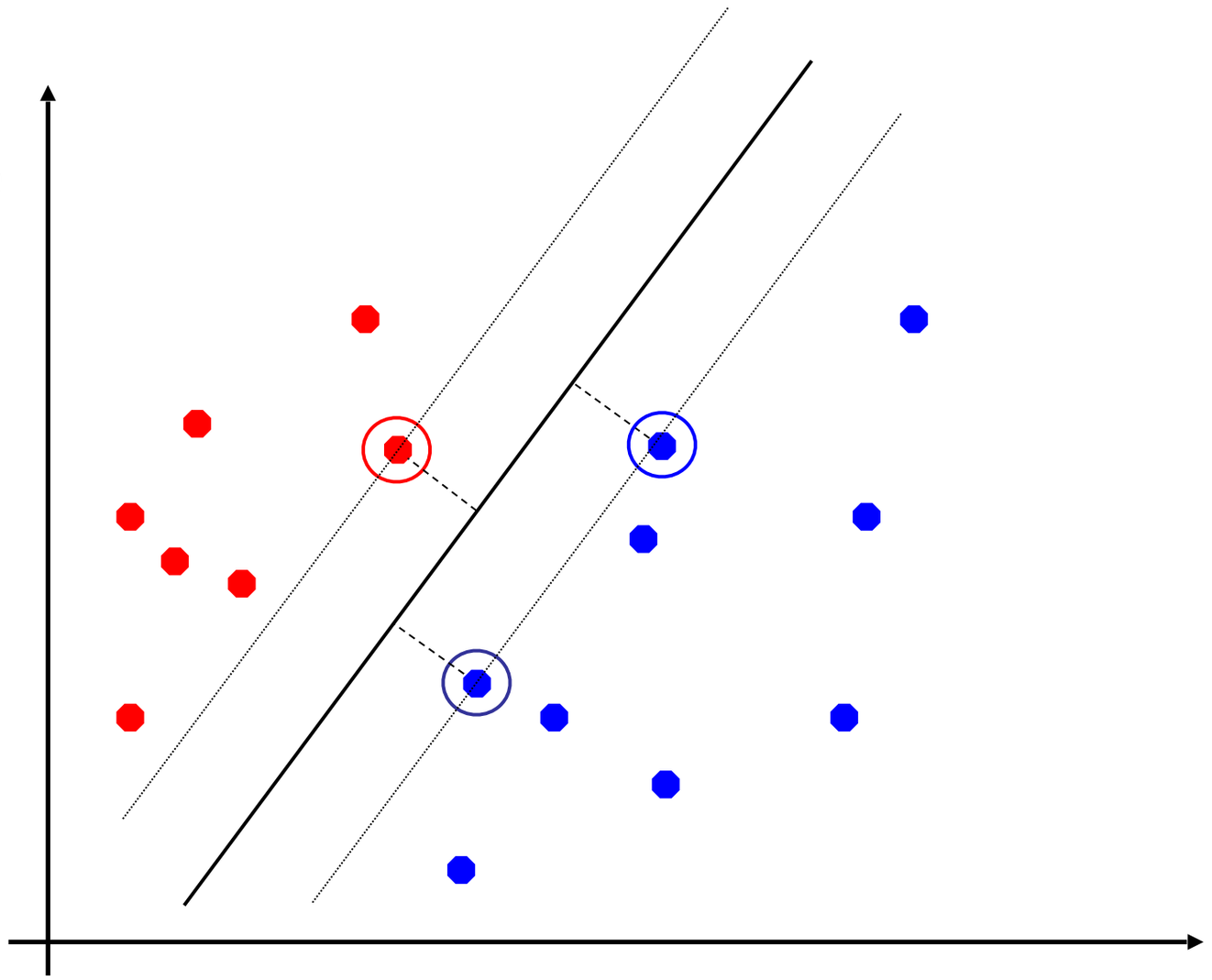


Which of these separators is optimal?

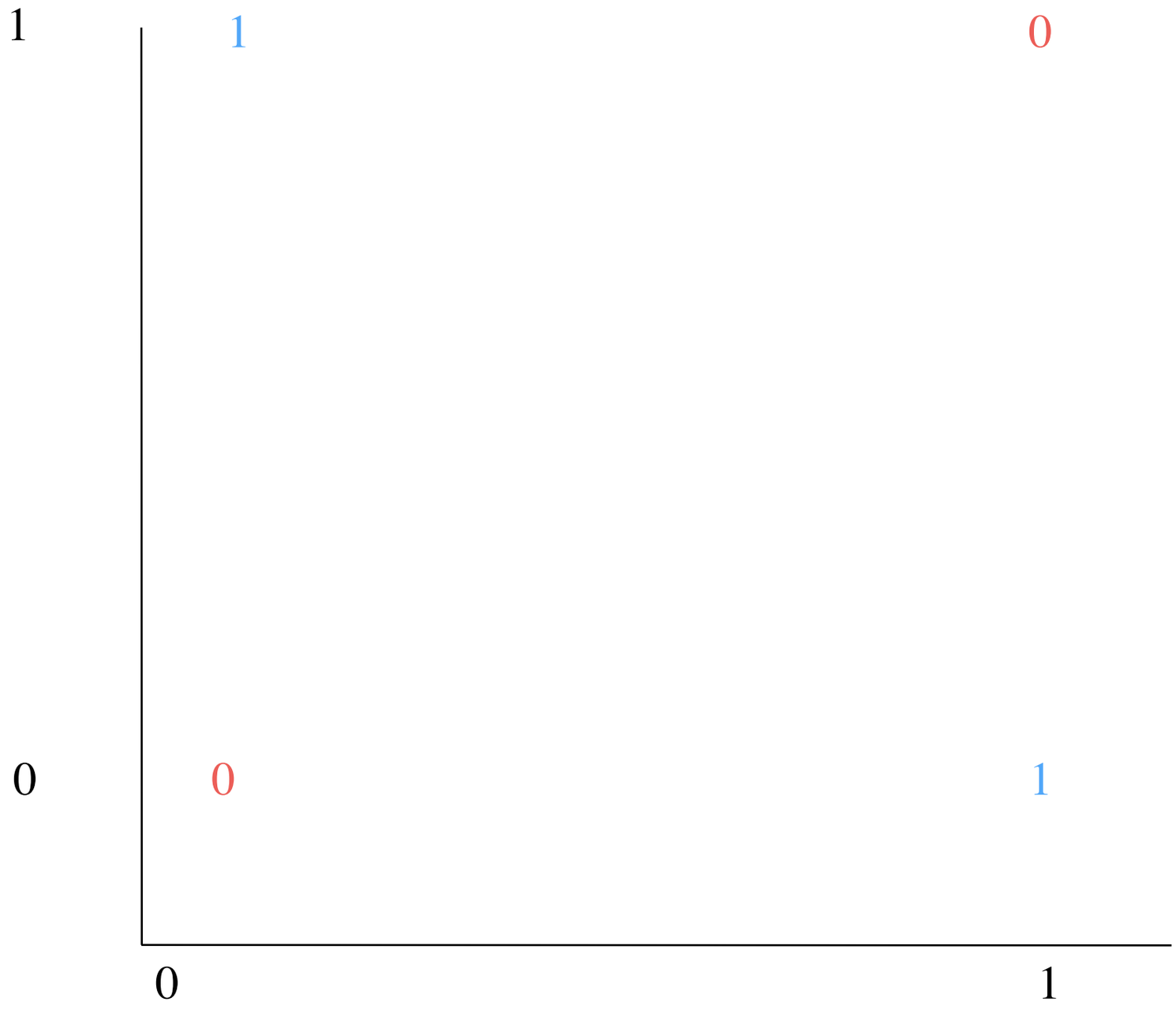


Support vector machines

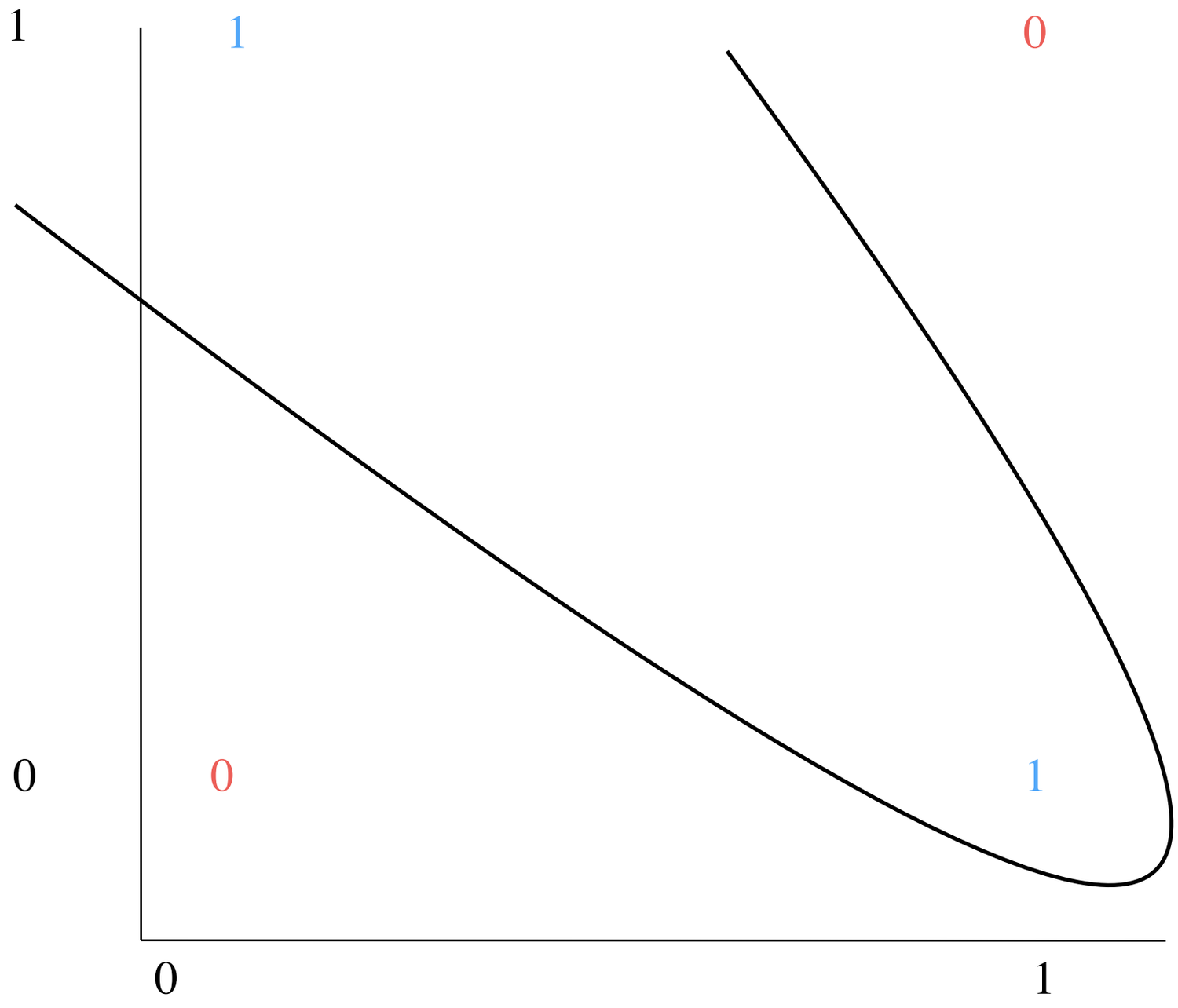
- Maximizing the margin: good according to intuition, theory, practice
- Only support vectors matter; other training examples are ignorable
- Support vector machines (SVMs) find the separator with max margin



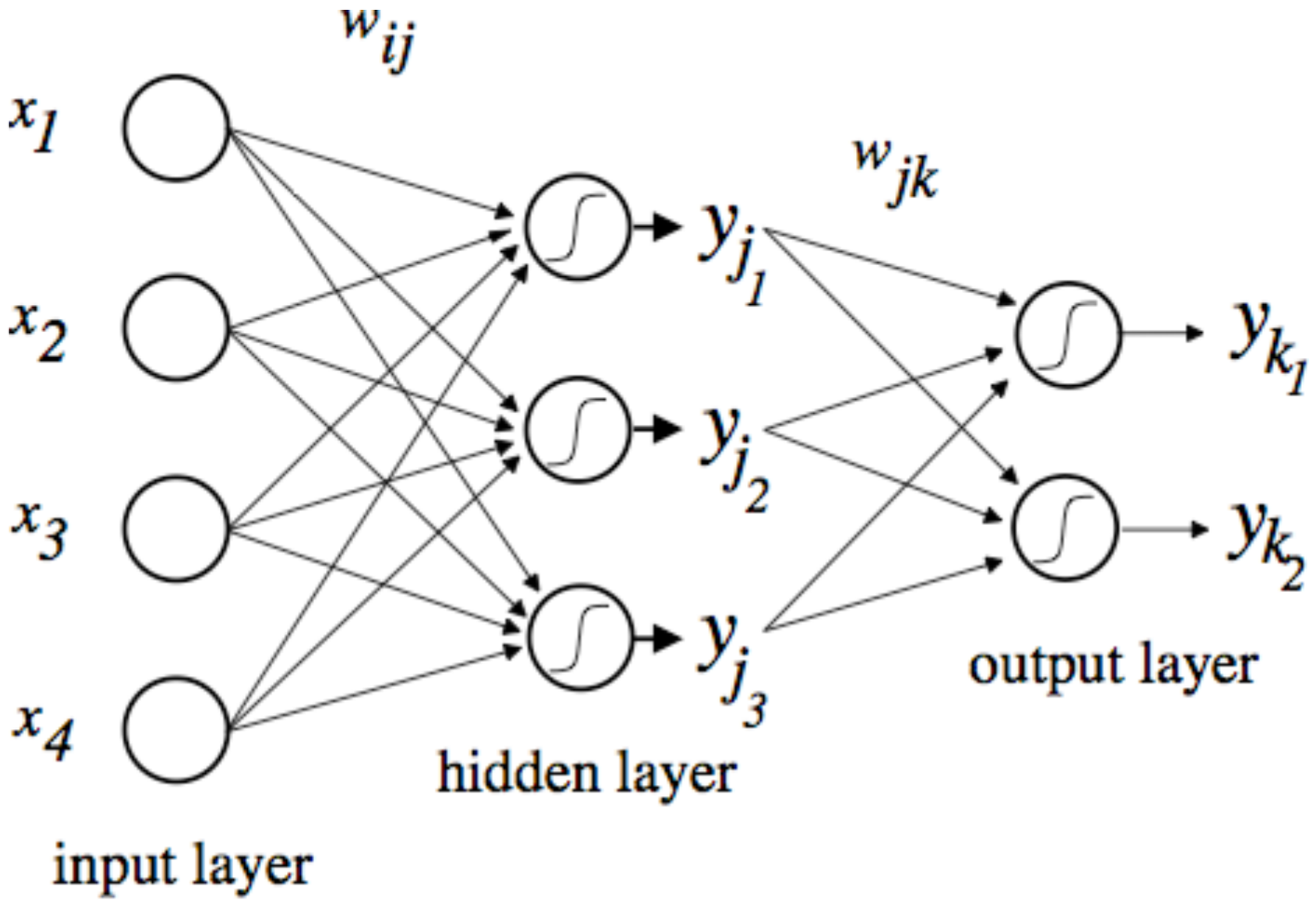
Problem: learn this!



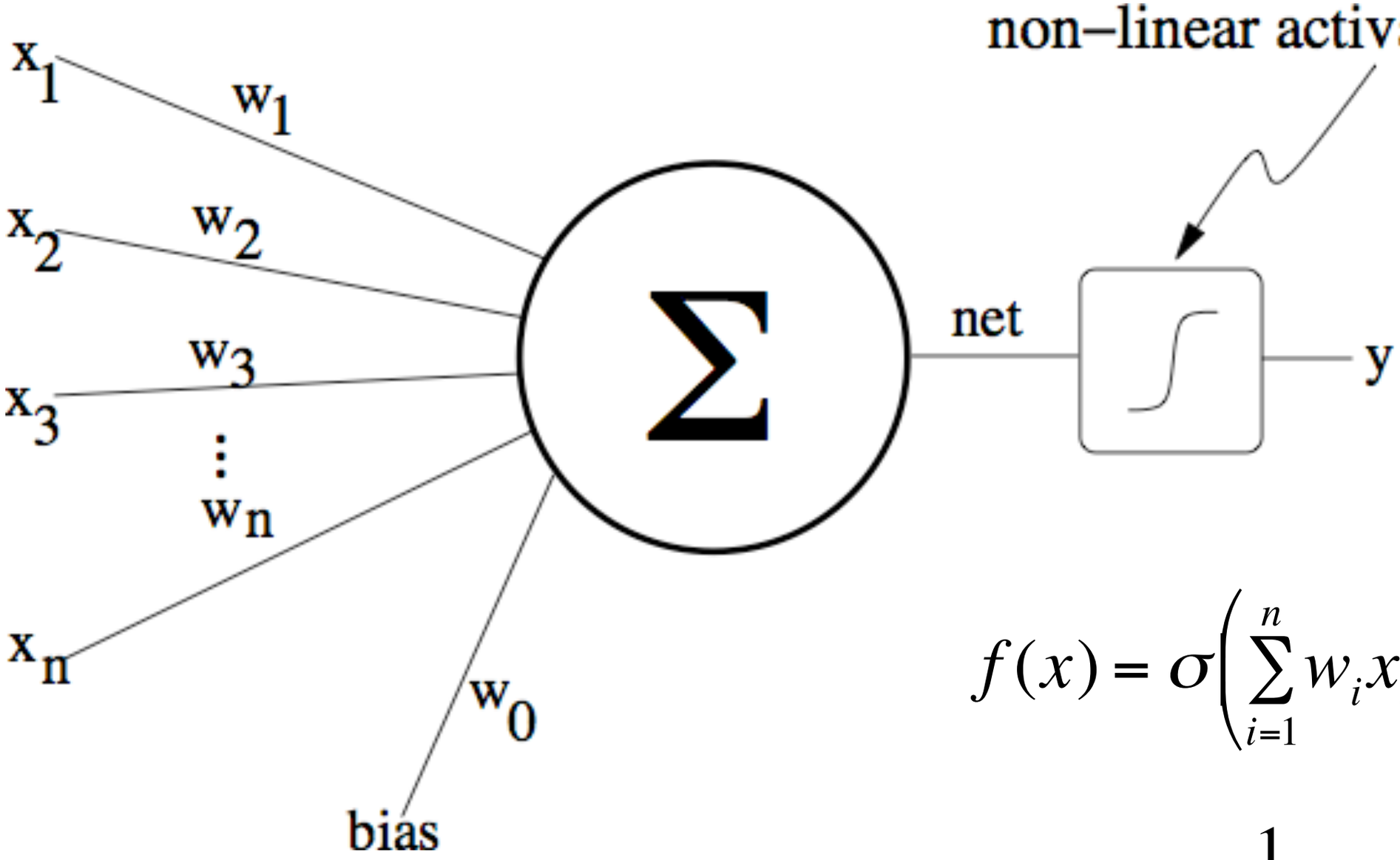
Problem: learn this!



Multi-layer Perceptron (MLP)



Non-Linear Neuron



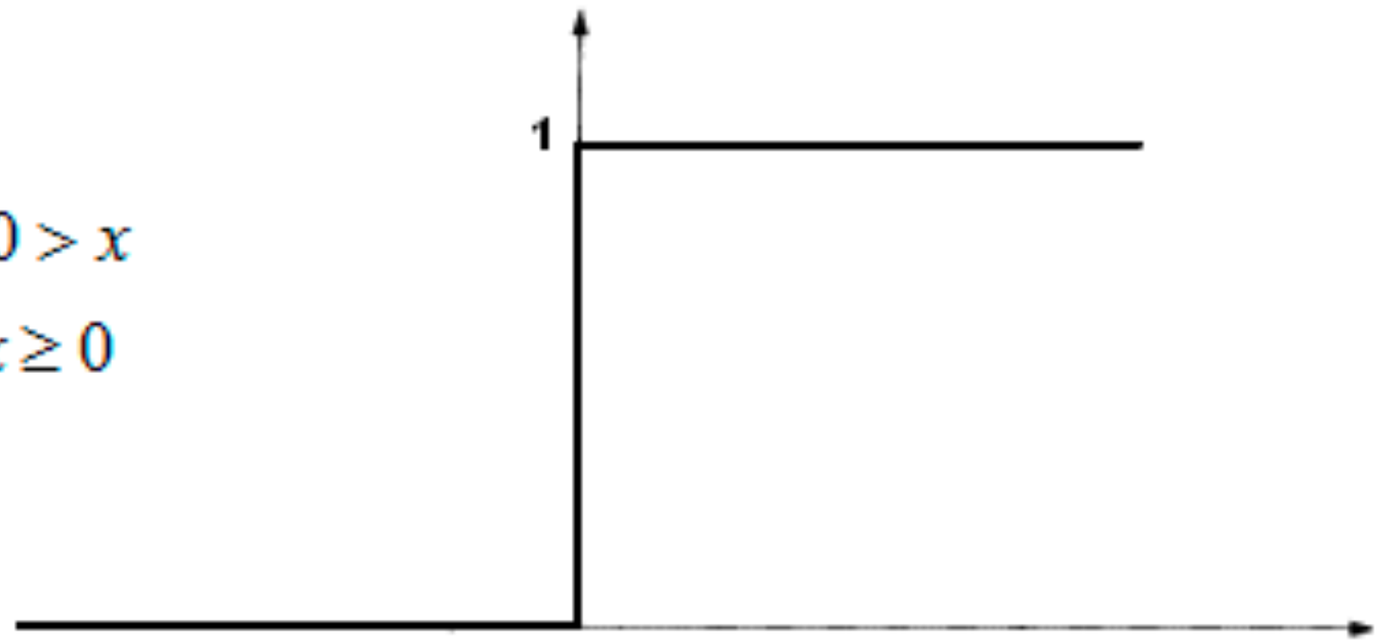
$$f(x) = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

$$\sigma(net) = \frac{1}{1+e^{-net}}$$

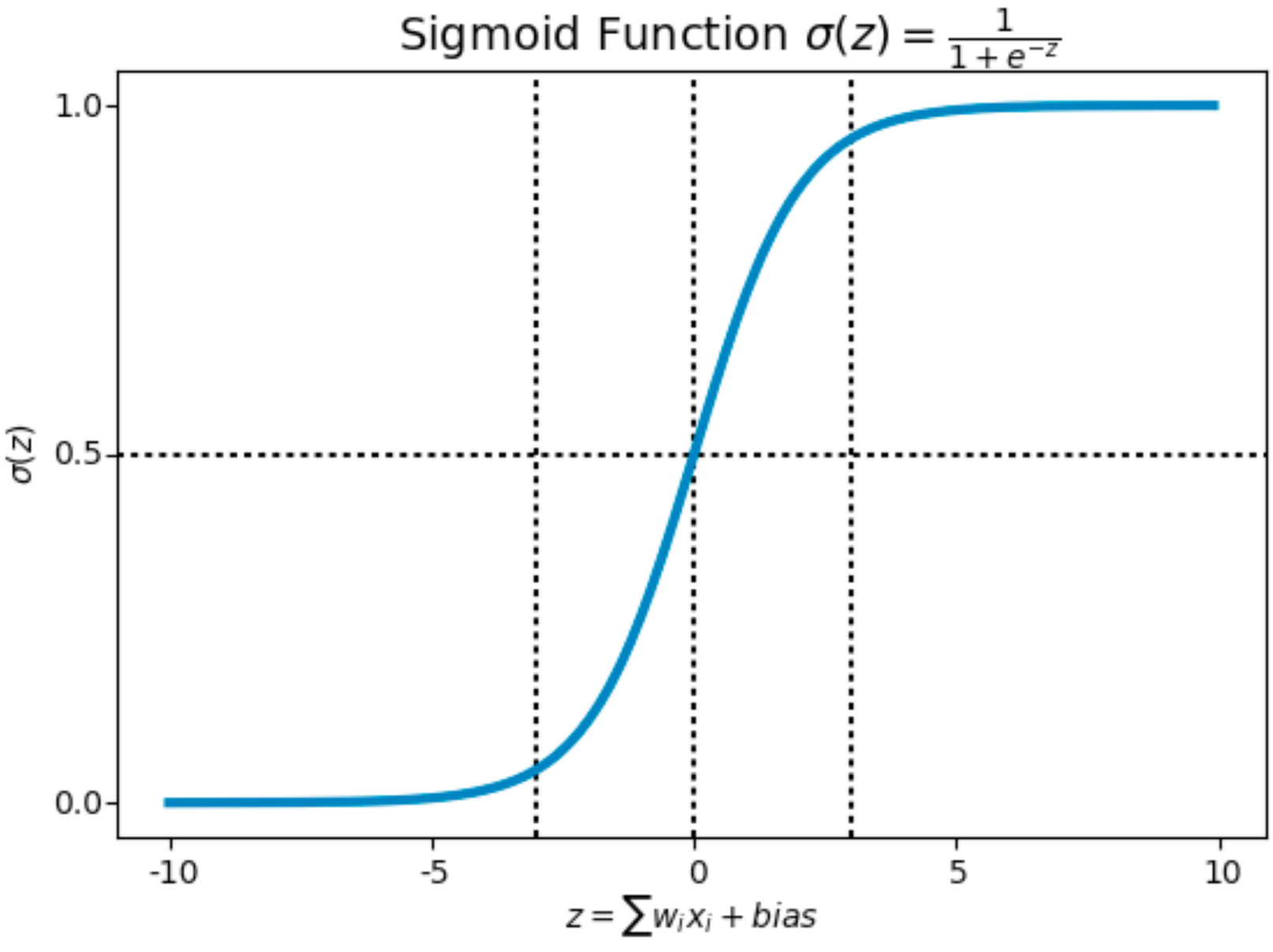
Activation Functions

Unit step (threshold)

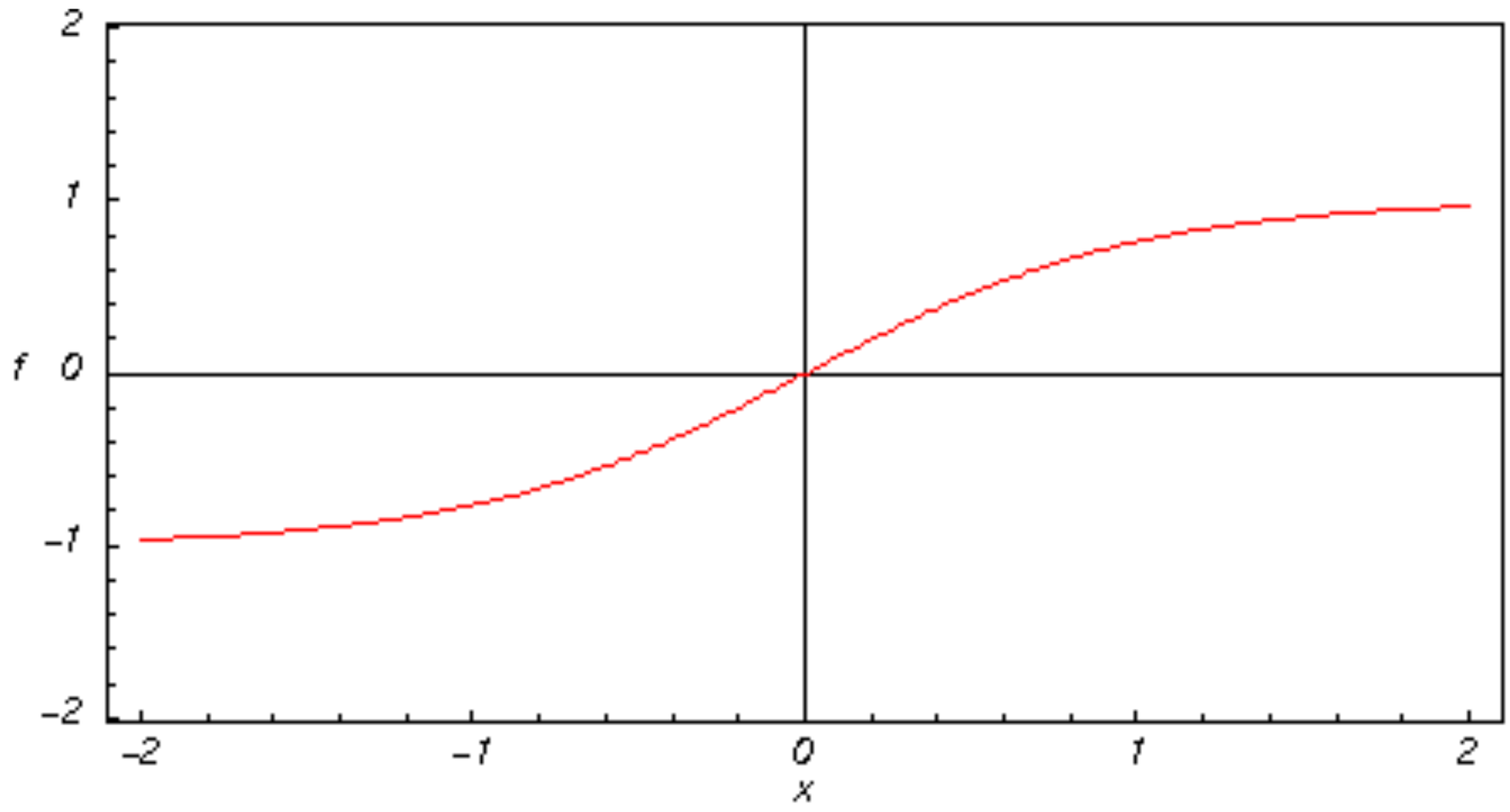
$$f(x) = \begin{cases} 0 \text{ if } 0 > x \\ 1 \text{ if } x \geq 0 \end{cases}$$



Activation Functions

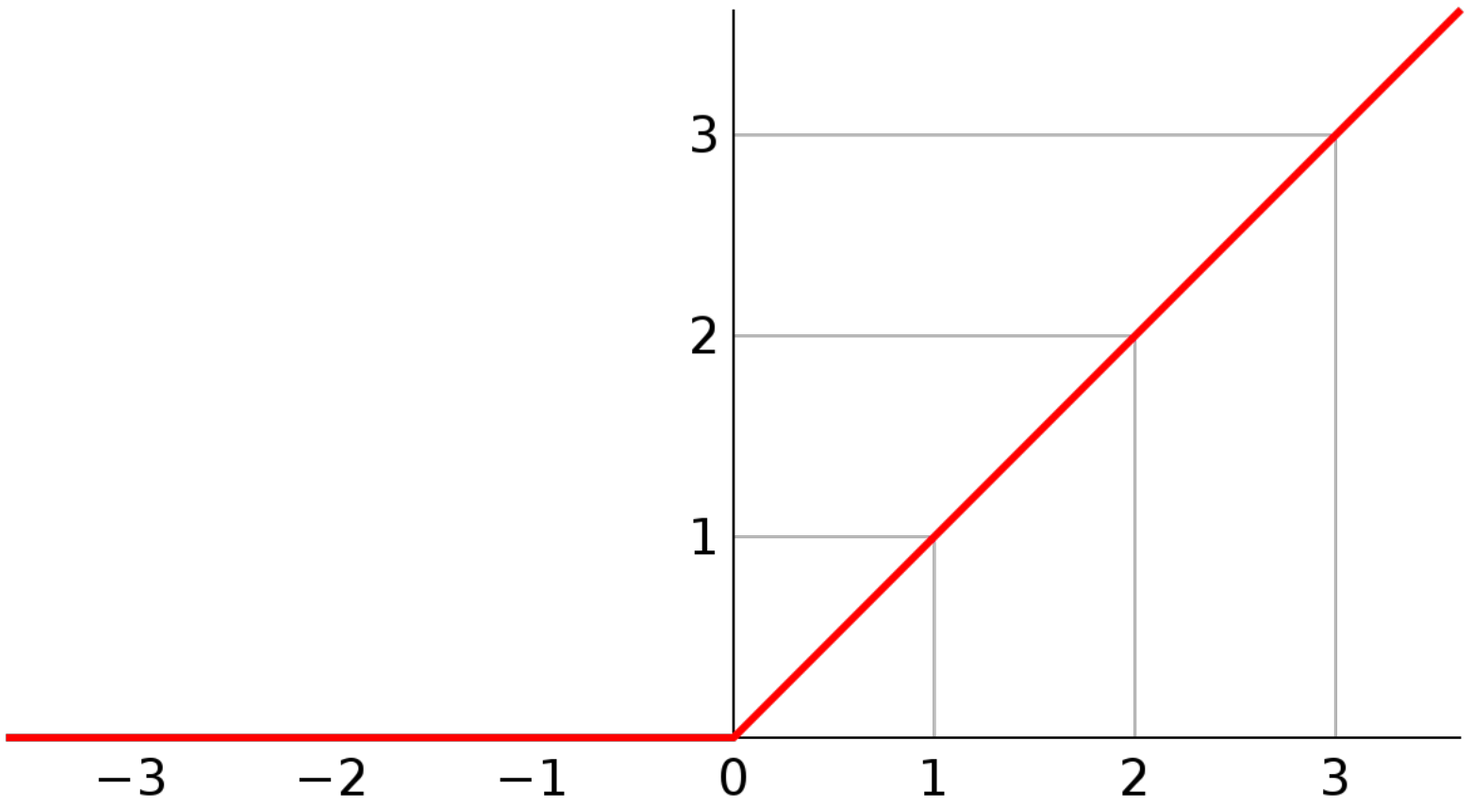


Activation Functions



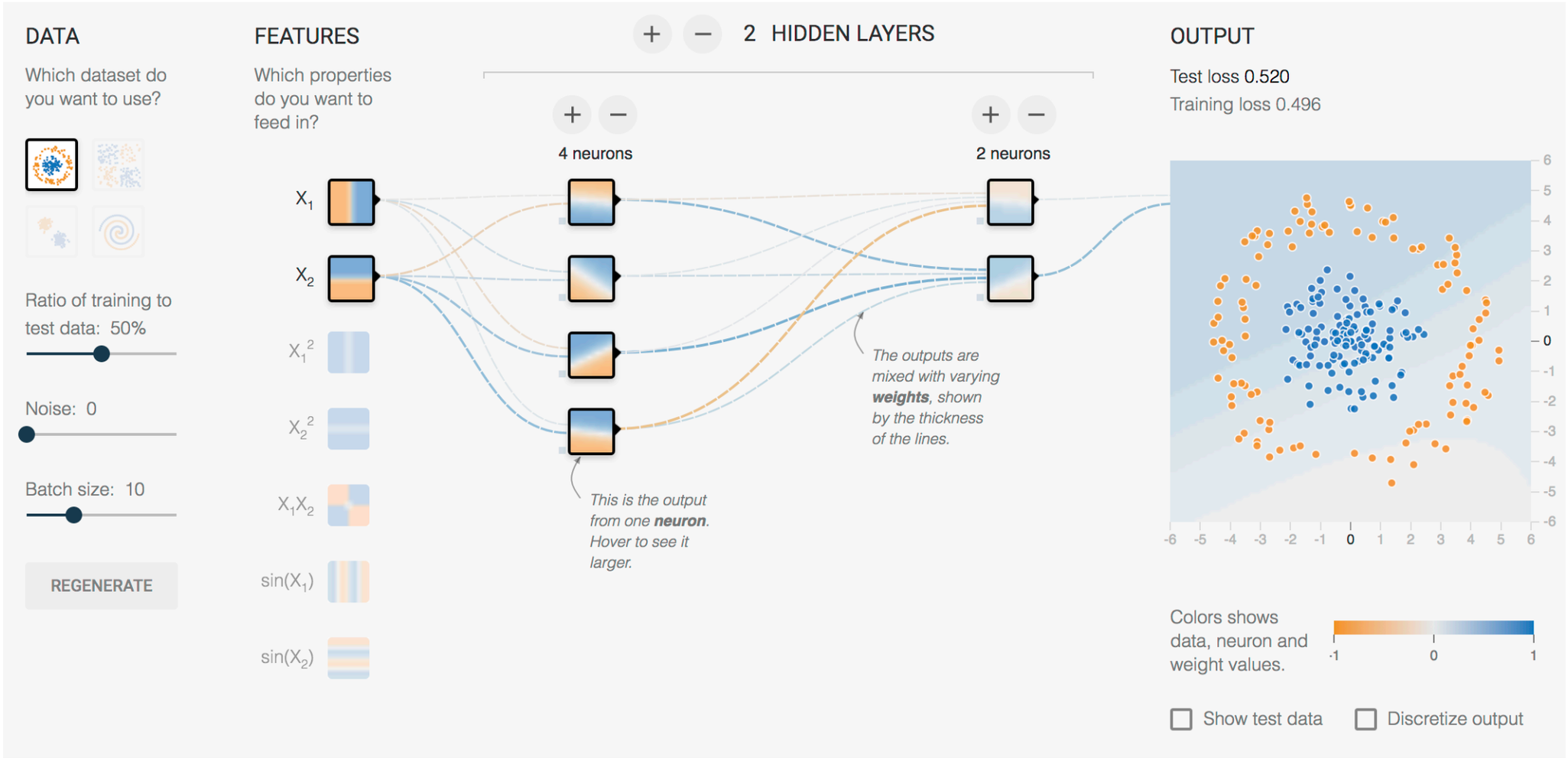
Hyperbolic Tangent

Activation Functions



Rectified Linear Units
(ReLu)

Elements of a Neural Network



http://playground.tensorflow.org/

Universal Approximator

- George Cybenko in 1989 provided one of the first proofs for the Universal Approximation Theorem for Neural Networks

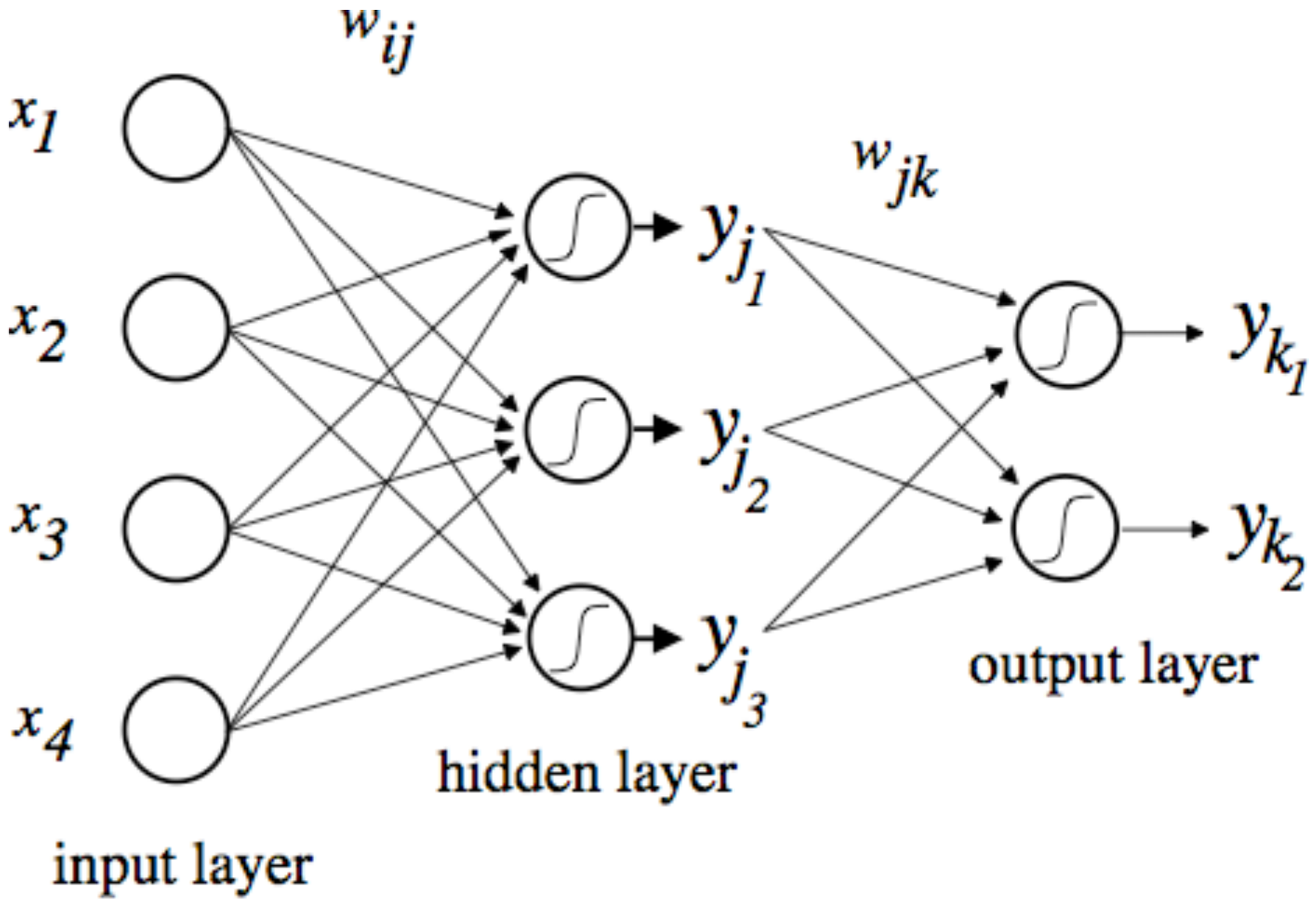
- States that a feedforward network with at least one hidden layer of a finite size can approximate any continuous function on compact subsets of R^n

- Says nothing about learnability

    - How do you train a hidden layer? What is its loss?

Training a Neural Network

Multi-layer Perceptron (MLP)



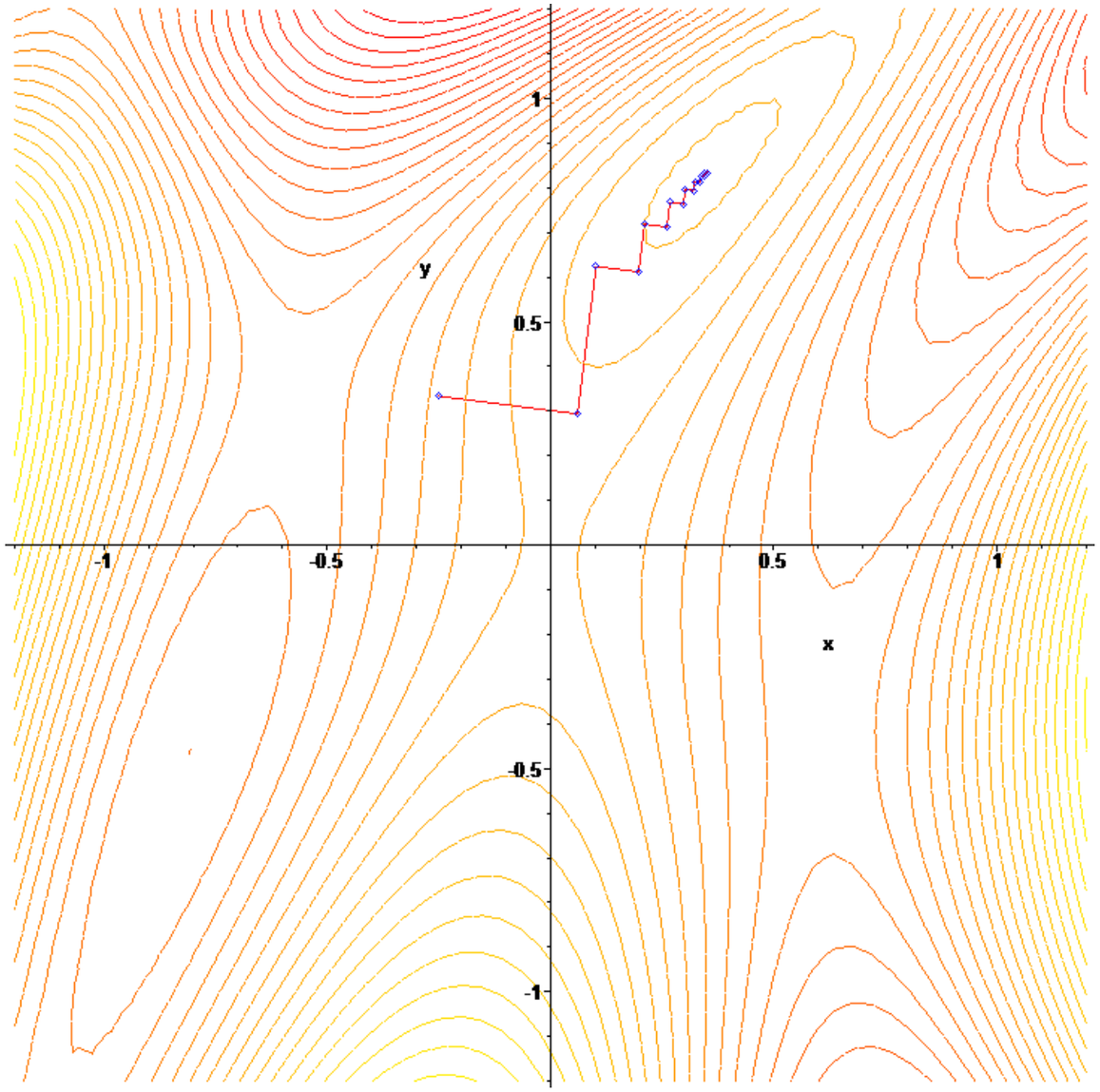
Backpropagation

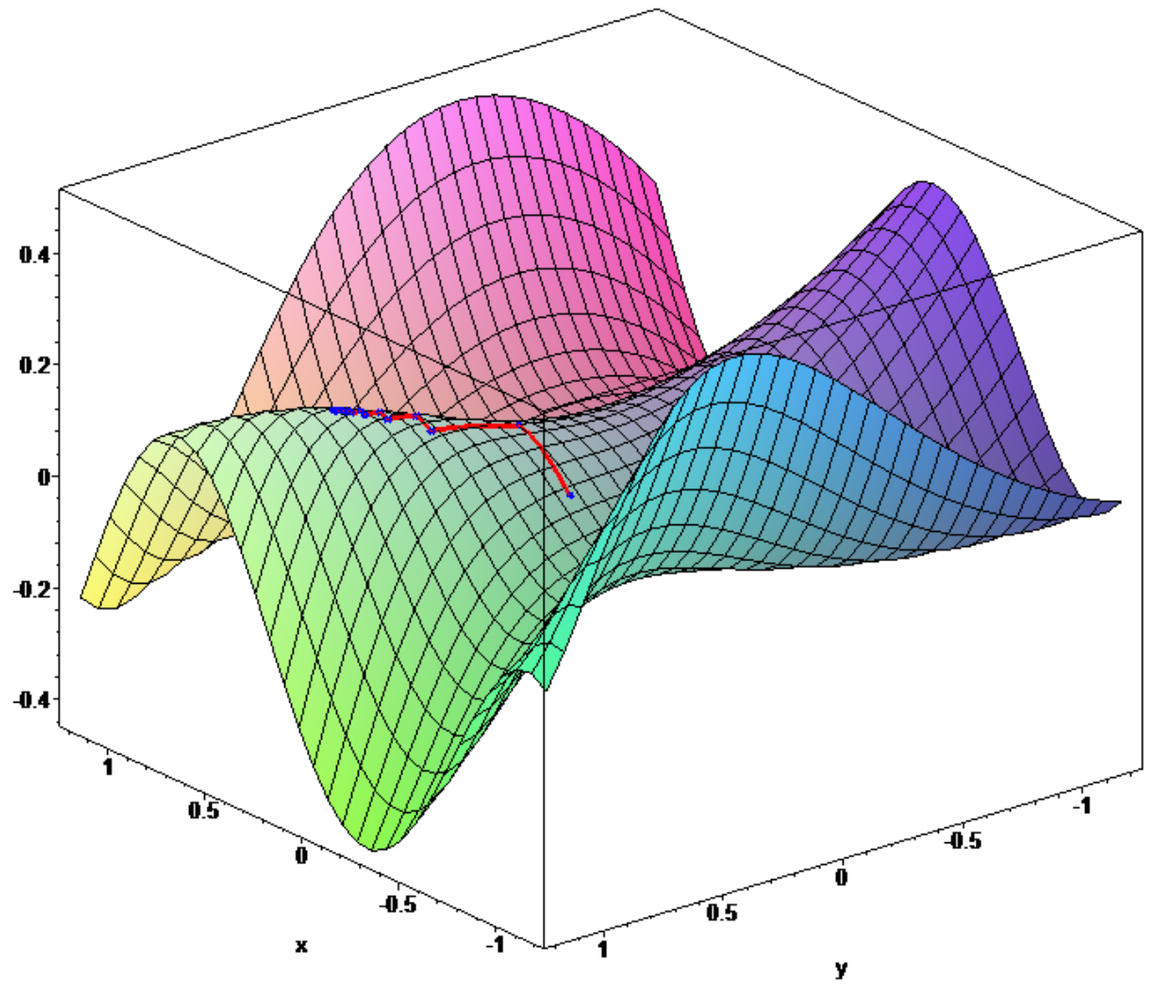
- Forward Pass: present training input pattern to network and activate network to produce output (can also do in batch: present all patterns in succession)

- Backward Pass: calculate error gradient and update weights starting at output layer and then going back

    - Modern libraries will compute the gradient for you!

Gradient Descent





Building a Neural Network

Weight Update Equation

$$\theta_w' = \theta_w - \eta * \frac{\partial TC}{\partial \theta_w}$$

New weight

Learning Rate